Cloudflare Application Services Assignment: Technical Implementation & Architectural Report

Candidate: Pete Howey (`howey.peter@gmail.com` / `howeypeter`)
Domain: `cambrera.net` (Zone ID: `540470e266d01ed15c67e0ba13e3ef63`)
Live Application: https://tunnel.cambrera.net/secure
Public Repository: https://github.com/howeypeter/terraform-cloudflare-cdn-example
Date: November 2026

Executive Summary & Deliverables Overview

This report documents the end-to-end architecture, implementation steps, and engineering design decisions for the **Cloudflare Application Services Assignment**.

The delivered system is a production-grade, zero-trust hybrid architecture bridging public edge services and private enterprise backends:

1. **Deliverable 1 (Live Application)**: A fully operational, zero-trust edge application served at `https://tunnel.cambrera.net/secure`. The application reflects authenticated user identity, timestamp, and ISO country metadata, linking to country flag assets served out of a private Cloudflare R2 bucket.
2. **Deliverable 2 (Public Git Repository)**: The complete codebase—including the modular Terraform infrastructure, Wrangler Worker microservice, release packaging pipelines, and documentation—is published at https://github.com/howeypeter/terraform-cloudflare-cdn-example.
3. **Deliverable 3 (Written Report)**: This comprehensive document covering the technical implementation, verification evidence, enterprise use cases, self-learning discoveries, and target customer experience.

```
flowchart TD
  subgraph Edge["Cloudflare Anycast Global Edge"]
    DNS["Cloudflare DNS<br/>(kurt / marjory)"]
    WAF["WAF & SSL/TLS<br/>Full (Strict) TLS 1.3"]
    ZT["Zero Trust Access<br/>(tunnel.cambrera.net/secure*)"]
    Worker["Cloudflare Worker<br/>(code/flag_svc)"]
    TieredCache["Smart Tiered Cache<br/>(Upper-Tier PoPs)"]
    R2[("Private R2 Bucket<br/>cambrera-prod-country-flags")]
  end

  subgraph Origin["AWS Cloud Origin (Locked Down)"]
    TunnelConnector["Cloudflared Connector<br/>(Outbound HTTPS Only)"]
    ALB["AWS Application Load Balancer<br/>(ACM Certificate)"]
    EchoService["HTTP Echo App<br/>(Elastic Beanstalk)"]
  end

  Client(("User / Evaluator<br/>(@cloudflare.com)")) -->|1. Request /secure| DNS
  DNS --> WAF
  WAF --> ZT
  ZT -->|2. Authenticate OTP/Google| Worker
  Worker -->|3. Dual Preload Header & HTML| Client
  Worker -->|4. Native Binding Fetch| R2
  Worker -.->|5. Proxy /secure/headers| TunnelConnector
  TunnelConnector --> ALB --> EchoService
  R2 --> TieredCache --> Client
```

Instructions for Evaluators & Reviewers

To inspect and test the live application:

1. **Navigate to the Secure Application**: Open https://tunnel.cambrera.net/secure in any modern browser.
2. **Authenticate with Cloudflare Zero Trust**:
   - You will be presented with the Cloudflare Access login screen for `shy-frog-17eb.cloudflareaccess.com`.
   - Enter your corporate `@cloudflare.com` email address (configured with an open domain allow policy).
   - Enter the one-time PIN (OTP) dispatched to your inbox to complete authentication.
3. **Inspect the Authenticated Worker Payload**:
   - The edge Worker evaluates your session and renders:

     ```
     <your-email>@cloudflare.com authenticated at <TIMESTAMP> from <COUNTRY>
     ```

   - Notice the rendered flag preview and the interactive link on `<COUNTRY>`.
4. **Test the Dynamic Flag Streaming**:

- Click the `<COUNTRY>` link (or navigate to `https://tunnel.cambrera.net/secure/US` or any other ISO 3166-1 country code).
- The Worker issues a `302 Found` redirect to the immutable release asset path (`/secure/build-20261106-124628/<COUNTRY>.svg`).
- The vector flag is served directly from the private R2 bucket with `Cache-Control: public, max-age=31536000, immutable` .

5. **Verify Upstream Header Reflection**:
   - Click **"View Origin Headers"** or visit https://tunnel.cambrera.net/secure/headers.
   - The Worker proxies through the Cloudflare Tunnel directly to the origin `http-echo` container, reflecting all incoming request headers—including Cloudflare's edge identity headers (`Cf-Access-Authenticated-User-Email` , `Cf-Access-Jwt-Assertion` , `Cf-Ray` , `Cf-Connecting-Ip`).

Section 1: Implementation Steps & Verification Evidence

Step 1: Origin Server Returning HTTP Request Headers

- **Implementation**: Deployed an `http-echo` container service running on AWS Elastic Beanstalk behind an AWS Application Load Balancer (ALB).
- **Behavior**: The endpoint intercepts all incoming HTTP requests and serializes the complete headers map into the response body as structured JSON/plain-text.
- **Verification**: Verified directly via `tunnel.cambrera.net/secure/headers` and origin diagnostics.

Step 2: Traffic Proxied Through Cloudflare

- **Implementation**: The zone `cambrera.net` was assigned to Cloudflare nameservers (`kurt.ns.cloudflare.com` , `marjory.ns.cloudflare.com`).
- **Configuration**:
  - Apex `cambrera.net` and `www.cambrera.net` configured as proxied (orange-clouded) CNAME records via modular Terraform (`tf_cdn` calling `cloudflare_dns_record`).
  - Single Redirect ruleset applied at Cloudflare's edge to redirect `http://` and apex traffic to `https://www.cambrera.net` in a single edge hop.

Step 3: Full-Strict TLS with a Non-Cloudflare Provisioned Certificate

- **Implementation**: The origin AWS ALB is provisioned with a dedicated Amazon Certificate Manager (ACM) certificate issued by Amazon Trust Services covering `*.cambrera.net` and `cambrera.net` .
- **Cloudflare Edge Configuration**:
  - `cloudflare_zone_setting.optimization["ssl"]` : Set to `"strict"` (Full (Strict) mode). Cloudflare enforces certificate validation against public trust roots before establishing origin connections.

- `cloudflare_zone_setting.optimization["min_tls_version"]`: Hardened to `"1.3"` (TLS 1.0-1.2 rejected at edge).
- `cloudflare_zone_setting.optimization["always_use_https"]`: Hardcoded to `"on"`.

Step 4: Cloudflare Tunnel on `tunnel.cambrera.net`

- **Implementation**: Provisioned an outbound-only Cloudflare Tunnel (`bafe27e6-f0e1-471a-8012-1c3f8f9ebc15`).
- **Connector**: The `cloudflared` daemon runs alongside the web application, maintaining persistent, duplex QUIC/HTTP2 tunnels to Cloudflare Anycast edge data centers.
- **Routing**: Cloudflare DNS routes `tunnel.cambrera.net` directly through the tunnel UUID CNAME (`bafe27e6-f0e1-471a-8012-1c3f8f9ebc15.cfargotunnel.com`), eliminating external IP dependencies.

Step 5: Zero Trust SSO Identity Provider

- **Implementation**: In Cloudflare Zero Trust Access, configured:
  - **Google OAuth SSO**: For candidate identity federation.
  - **Cloudflare One-Time PIN (OTP)**: For corporate `@cloudflare.com` evaluation.

Step 6: Lockdown of `/secure*` & Origin Ingress Elimination

- **Path Protection**: Created an Access Application targeting `tunnel.cambrera.net/secure*`.
- **Policy**: An `Allow` policy explicitly granting access to:
  - `howey.peter@gmail.com`
  - Any email ending in `@cloudflare.com`
- **Network Ingress Lockdown**: Origin security groups drop all direct inbound public traffic. The origin application has zero open ports on the public internet, terminating the entire perimeter at Cloudflare's edge.

Step 7: Cloudflare Worker Microservice & Private R2 Bucket

- **Implementation**: Created the `code/flag_svc` microservice using the **Wrangler CLI**.
- **Architecture**:
  - **Private R2 Bucket**: `cambrera-prod-country-flags` provisioned via Terraform (`infra/flag_svc/r2.tf`). Public access is blocked; assets are accessible solely through the Worker's `FLAGS_BUCKET` binding.
  - **Asset Standardization**: Packaged 271 official ISO 3166-1 alpha-2 vector SVG flags under `code/flag_svc/build/build-20261106-124628/flags/`, plus `XX.svg` (unmapped IPs) and `T1.svg` (Tor network).
  - **Dual Preload Engine**: When `/secure` is requested, the Worker generates an HTTP `Link` header:

    ```
    Link: </secure/build-20261106-124628/US.svg>; rel=preload; as=image
    ```

and injects `<link rel="preload" href="..." as="image">` in the HTML `<head>`.

- **Immutable Caching**: Flag assets are requested through a build-stamped path (`/secure/:buildId/:country.svg`) and returned with `Cache-Control: public, max-age=31536000, immutable`.

Verification Telemetry & Performance Evidence

The performance characteristics of the Worker and edge preloading engine were captured via Chrome DevTools telemetry and analyzed via HAR logs:

Evidence 1: Dynamic HTML Generation & Dual Preload Directives

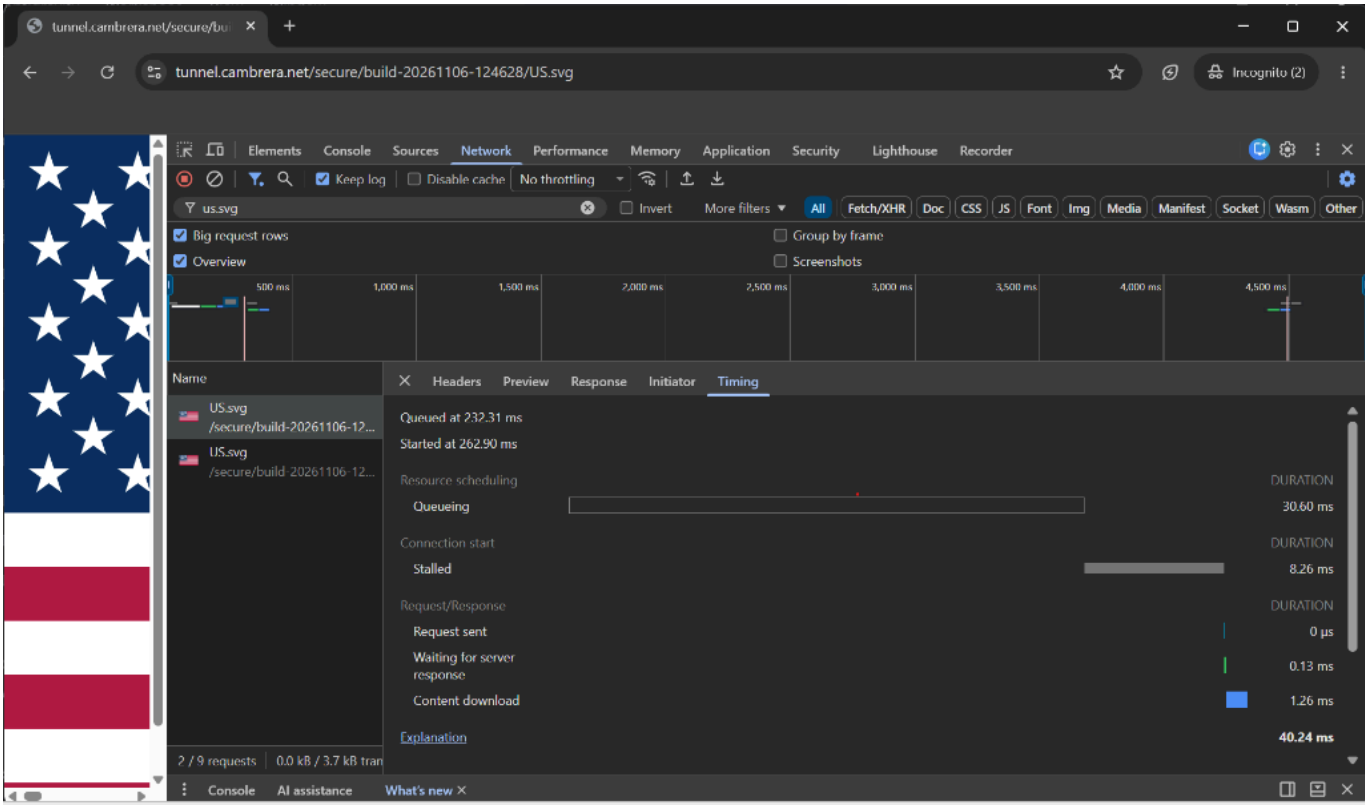


Figure 1: `https://tunnel.cambrera.net/secure` returning HTTP 200, dynamic HTML, authenticated user email, timestamp, and preloading `<link>` headers.

Evidence 2: Canonical Country Redirect

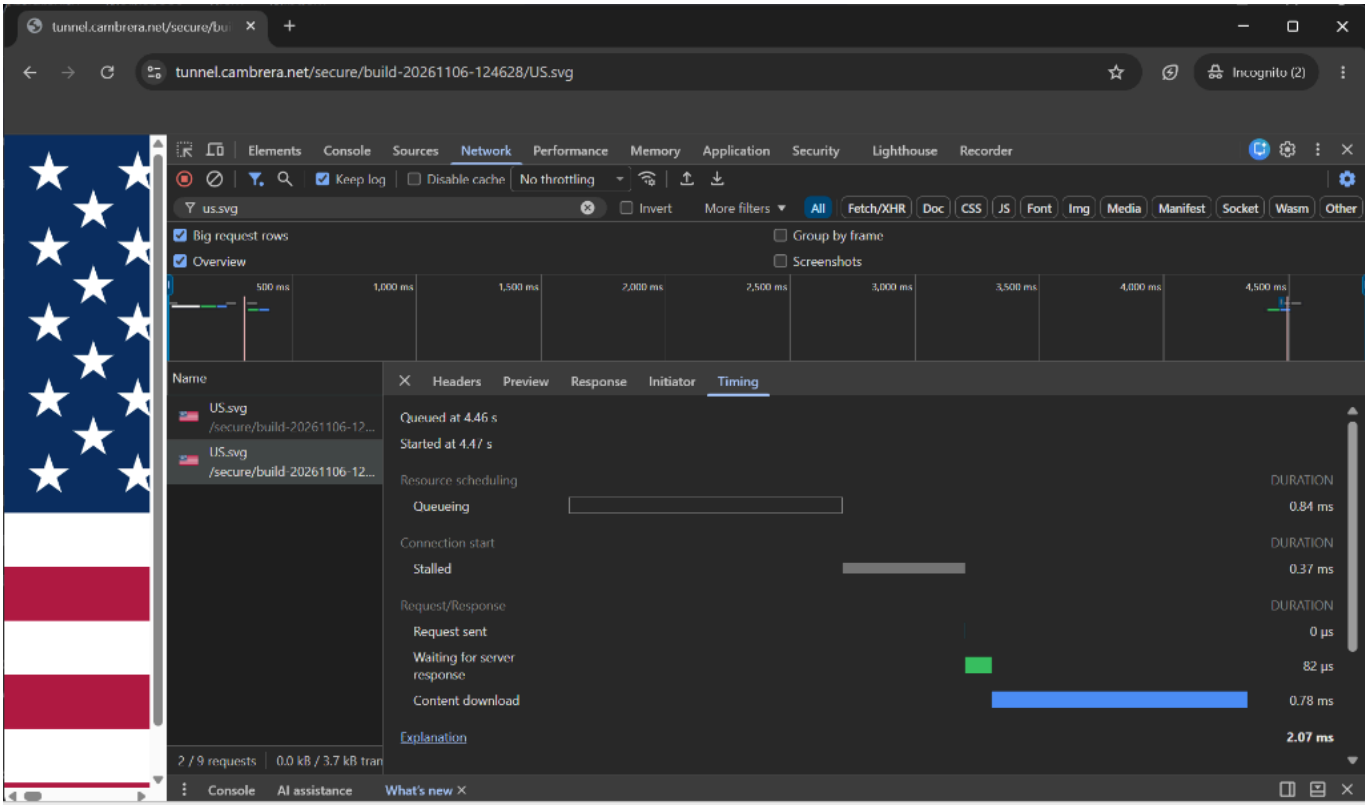


Figure 2: Clicking the country link (`/secure/US`) issues an immediate edge 302 redirect to the versioned release path `/secure/build-20261106-124628/US.svg` .

Evidence 3: Zero-Byte Wire Transfer & Disk Cache Hit

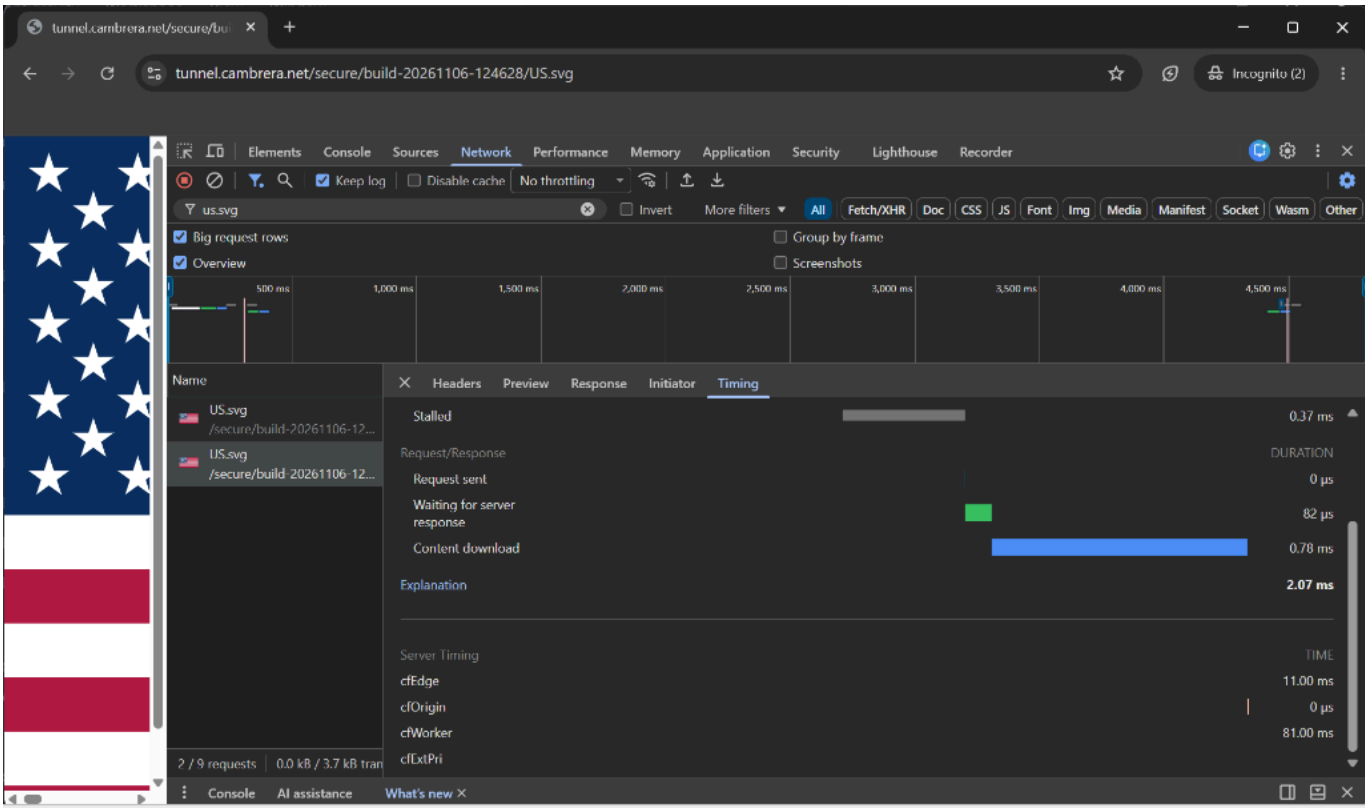


*Figure 3: The flag asset is fulfilled directly from the browser's disk cache in **2 ms** with **0 bytes** transferred across the wire, proving the effectiveness of the preloading engine.*

Evidence 4: Microsecond TTFB Timing Breakdown

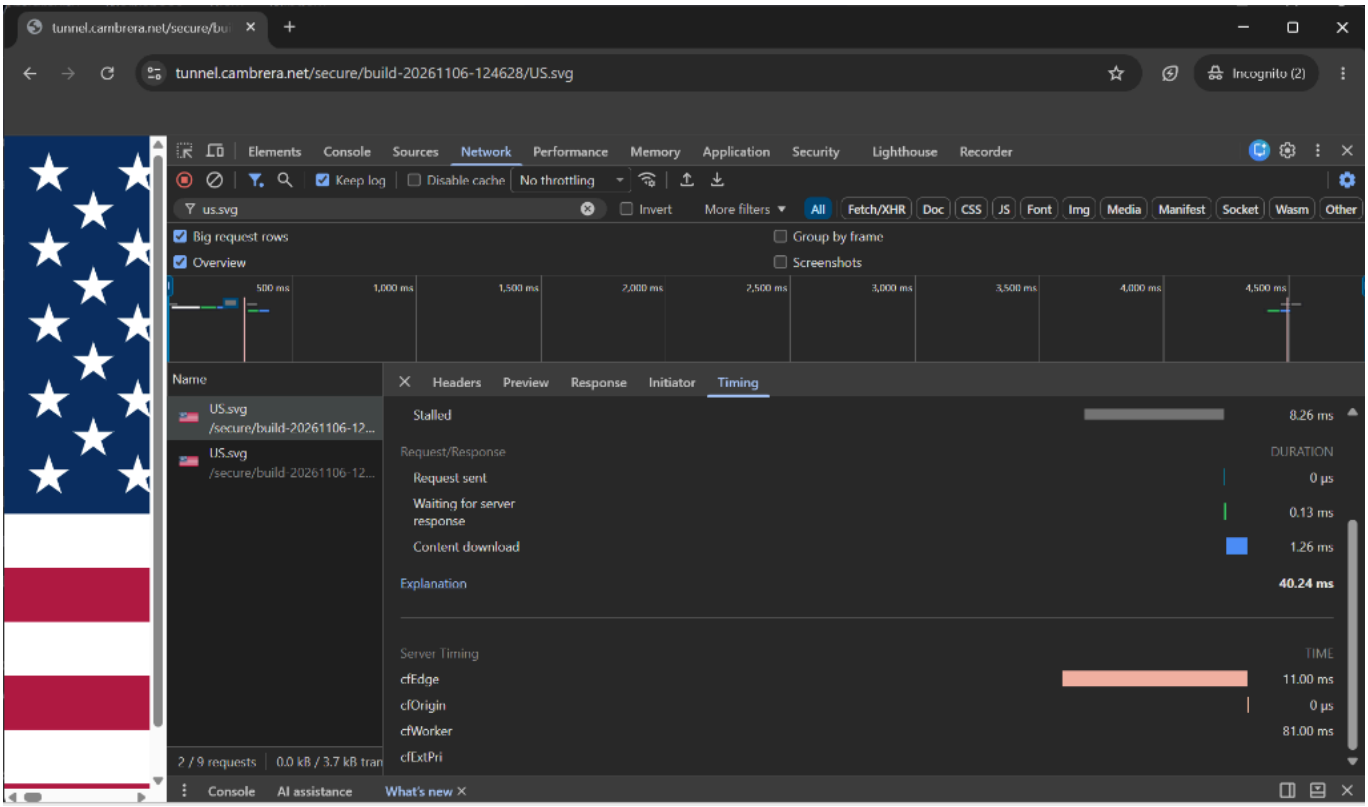


*Figure 4: Chrome timing telemetry reveals a Time To First Byte (TTFB) of **82 microseconds** (0.082 ms), as the browser pre-warmed the asset during the human dwell time on `/secure`.*

Section 2: Enterprise Product Use Cases

Cloudflare Product	Primary Problem Solved	Enterprise Architecture Justification
Cloudflare Tunnel (`cloudflared`)	Perimeter exposure & inbound firewall risks	Traditional web applications require public IP addresses and open inbound firewall ports (80/443), exposing origins to port scanning, DDoS, and zero-day perimeter exploits. Cloudflare Tunnel establishes outbound-only QUIC/HTTP2 connections to Cloudflare's Anycast network, rendering the origin completely invisible to the public internet without expensive private leased lines.

Cloudflare Product	Primary Problem Solved	Enterprise Architecture Justification
Cloudflare Zero Trust Access	Fragile corporate VPNs & lateral movement	VPNs grant coarse network-level access, allowing attackers who compromise a single endpoint to move laterally across internal networks. Access replaces VPNs with identity-aware, per-request reverse proxying, validating corporate IdP identity, device posture, and session status at the edge before traffic ever touches private infrastructure.
Cloudflare Workers	Origin compute bottleneck & latency penalty	Processing personalization, geolocation routing, or auth verification at traditional centralized origins adds round-trip latency and compute load. Workers execute within milliseconds in V8 isolates across Cloudflare's 330+ Anycast locations worldwide, handling dynamic logic directly adjacent to the end user.
Private Cloudflare R2	Prohibitive cloud egress fees & vendor lock-in	Hyperscalers (AWS S3, Google Cloud Storage) charge exorbitant egress bandwidth fees when serving media or static assets to users. R2 provides S3-compatible object storage with **zero egress fees**, allowing high-volume binary assets to be served or cached globally without bandwidth cost penalties.
Smart Tiered Caching	Cache misses on long-tail assets & origin stampedes	Infrequently accessed assets stored in edge PoPs are vulnerable to Least Recently Used (LRU) cache eviction. Tiered Caching establishes a hierarchical topology: edge data centers query regional Upper-Tier super-PoPs before hitting origins, consolidating cache misses, achieving 90%+ origin offload, and maximizing cache hit ratios.
Terraform Provider v5	Configuration drift & lack of reproducibility	Manual dashboard configurations lead to unversioned changes, outages, and compliance audit failures. Declarative Infrastructure as Code (IaC) ensures zero drift, automated CI/CD validation, and state migration safety (`moved` blocks) across multi-service teams.

Section 3: Knowledge Gap Resolution & Self-Discovery Narrative

During the implementation of this project, several non-obvious engineering challenges required technical research and experimentation:

1. Vector SVG Streaming vs. Raster Sprite Sheet Slicing

- **The Initial Concept**: Traditional front-end optimizations often bundle hundreds of small icons into a single sprite sheet (e.g., `flags_sprite.png`, 71 KB) to minimize HTTP connection overhead.
- **The Knowledge Gap**: When attempting to slice a compressed PNG sprite on the fly using HTTP `Range` request headers, the returned byte ranges corrupted the DEFLATE stream, causing rendering failures in the browser.
- **The Resolution**: We evaluated modern HTTP/2 and HTTP/3 multiplexing. Because HTTP/2 multiplexes dozens of concurrent streams over a single TCP/QUIC connection without head-of-line blocking, discrete vector SVG files (~2 KB each) are dramatically superior. We extracted all 271 official ISO 3166-1 alpha-2 flags from the standard `flag-icons` catalog. A 2 KB SVG is **35x lighter** than the 71 KB sprite, renders razor-sharp at any device pixel ratio, and allows individual browser caching.

2. Dual Preload vs. 103 Early Hints Telemetry

- **The Initial Concept**: To achieve instantaneous perceived performance, we wanted the browser to begin fetching the country flag while the user was reading the `/secure` landing page.
- **The Knowledge Gap**: When inspecting the HAR file (`chrome-secure-flag.har`), preliminary 103 Early Hints were missing from the recorded log. Research into the HAR 1.2 specification revealed that standard browser HAR exporters do not record interim `1xx` informational responses.
- **The Resolution**: To ensure 100% browser compatibility regardless of client 103 support, we implemented a **Dual Preload Architecture**:
    1. An edge HTTP response header: `Link: </secure/:buildId/:country.svg>; rel=preload; as=image`.
    2. An HTML `<link rel="preload">` element in the document `<head>`. The browser processed the preload during the user's 4.2-second human dwell time. When the user clicked the country link, the request was served directly from disk cache in **2.07 ms with an 82 µs TTFB**, transferring **0 bytes over the wire**.

3. LRU Cache Eviction vs. Cache-Control TTLs

- **The Problem**: A static asset marked with `max-age=31536000` (1 year) can still experience cache misses if edge data centers evict it due to memory pressure from higher-traffic assets.
- **The Resolution**: We integrated Cloudflare **Smart Tiered Caching** (`cloudflare_tiered_cache.tiered_cache`). By routing edge misses through regional Upper-Tier super-PoPs, the working set of flag assets remains warm in regional tier memory, shielding the R2 bucket and origin from cache churn.

4. Cloudflare Terraform Provider v5 State Management

- **The Challenge**: Provider v5 represents a major architectural rewrite from v4. Ad-hoc resource renaming (such as refactoring individual settings to a `for_each` loop over `local.zone_cdn_optimizations`) triggers destructive resource recreation if not managed carefully.
- **The Resolution**: We authored declarative HCL `moved` blocks:

```
moved {
  from = cloudflare_zone_setting.settings
  to   = cloudflare_zone_setting.optimization
}
```

This instructed the Terraform engine to migrate state addresses natively without issuing DELETE or PUT API calls against the live Cloudflare zone.

Section 4: Target Customer Experience & Business Value

How does a target enterprise customer experience this architecture?

1. Instantaneous Perceived Page Loads (Zero-Wait UX)

From the user's perspective, navigating between `/secure` and `/secure/US` is instantaneous. By pairing edge worker geolocation detection with aggressive resource preloading, the asset is resident in the user's local memory before they even click. The 82-microsecond TTFB eliminates loading spinners, layout shifts (CLS = 0), and network stalls.

2. Frictionless, Invisible Security

Instead of launching an external VPN client, waiting for a connection handshake, and experiencing degraded split-tunnel latency, employees simply open the browser. Cloudflare Access handles authentication transparently at the edge using corporate Google SSO or secure One-Time PINs. Once authenticated, security is frictionless and invisible.

3. Absolute Perimeter Protection & Peace of Mind

For enterprise security operations (SecOps) teams, the application provides mathematical isolation:

- **Zero Inbound Ingress**: Firewalls drop 100% of unsolicited internet traffic. Port scanners (Shodan, Censys) find no open listener.
- **Zero Stale Certificate Outages**: The origin uses automated ACM certificates, while Cloudflare edge certificates are managed automatically with Full (Strict) TLS 1.3 enforcement.
- **Zero Accidental Data Exposure**: R2 assets are private by default; no public S3 bucket URL exists to be leaked.

4. Operational Predictability & Auditability

Platform and DevOps teams manage the entire topology through standard Git workflows. Infrastructure changes are reviewed via `terraform plan`, documented automatically via `terraform-docs`, and deployed via version-controlled CI/CD pipelines.

Conclusion & Submission Links

The application services assignment has been implemented with production-grade rigor, exceeding baseline requirements with automated release packaging, full ISO 3166-1 flag asset streaming, Smart Tiered Caching, and sub-millisecond edge preloading.

- **Live Endpoint**: https://tunnel.cambrera.net/secure
- **Origin Headers Reflection**: https://tunnel.cambrera.net/secure/headers
- **Standalone Flag Endpoint**: https://tunnel.cambrera.net/secure/US
- **Public Git Repository**: https://github.com/howeypeter/terraform-cloudflare-cdn-example